

Go Emotions Analysis

```
In [1]: import pandas as pd
import numpy as np
import re
from collections import Counter
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
import matplotlib.pyplot as plt
import seaborn as sns
G_train = pd.read_csv("GoEmotions/Processed/GoEmotions_train.csv")
G_test = pd.read_csv("GoEmotions/Processed/GoEmotions_test.csv")

G_train = G_train.rename(columns={'0': "text", '1': "label_id", '2': "example_id"})
G_test = G_test.rename(columns={'0': "text", '1': "label_id", '2': "example_id"})
```

```
In [178... G_emotion_cols = [
    "admiration", "amusement", "anger", "annoyance", "approval", "caring",
    "confusion", "curiosity", "desire", "disappointment", "disapproval",
    "disgust", "embarrassment", "excitement", "fear", "gratitude",
    "grief", "joy", "love", "nervousness", "optimism", "pride",
    "realization", "relief", "remorse", "sadness", "surprise", "neutral"
]

def parse_labels(label_str):
    return [int(x) for x in str(label_str).split(",")]

def make_multilabel_columns(df):
    df = df.copy()

    # Convert "13,18" into [13, 18]
    df["label_list"] = df["label_id"].apply(parse_labels)

    # Add readable labels: [13,18] -> ["excitement", "love"]
    df["label_names"] = df["label_list"].apply(
        lambda ids: [G_emotion_cols[i] for i in ids]
    )

    # Create one binary column per emotion
    for i, emotion in enumerate(G_emotion_cols):
        df[emotion] = df["label_list"].apply(lambda labels: 1 if i in labels else 0)

    return df

G_train = make_multilabel_columns(G_train)
G_test = make_multilabel_columns(G_test)
```

```
# print(goemotions_train[["label_id", "label_list", "label_names"]].head())

print(G_train.columns)
print(G_test.columns)
```

```
Index(['text', 'label_id', 'example_id', 'label_list', 'label_names',
      'admiration', 'amusement', 'anger', 'annoyance', 'approval', 'carin
g',
      'confusion', 'curiosity', 'desire', 'disappointment', 'disapproval',
      'disgust', 'embarrassment', 'excitement', 'fear', 'gratitude', 'grie
f',
      'joy', 'love', 'nervousness', 'optimism', 'pride', 'realization',
      'relief', 'remorse', 'sadness', 'surprise', 'neutral', 'num_labels',
      'word_count', 'clean_word_count', 'char_count'],
      dtype='str')
Index(['text', 'label_id', 'example_id', 'label_list', 'label_names',
      'admiration', 'amusement', 'anger', 'annoyance', 'approval', 'carin
g',
      'confusion', 'curiosity', 'desire', 'disappointment', 'disapproval',
      'disgust', 'embarrassment', 'excitement', 'fear', 'gratitude', 'grie
f',
      'joy', 'love', 'nervousness', 'optimism', 'pride', 'realization',
      'relief', 'remorse', 'sadness', 'surprise', 'neutral'],
      dtype='str')
```

Here is a method that can be used for cleaning text and finding the most common words.

```
In [179... custom_stopwords = set(ENGLISH_STOP_WORDS).union({
    "im", "ive", "dont", "didnt", "doesnt", "cant", "couldnt",
    "wouldnt", "isnt", "arent", "wasnt", "werent",
    "feel", "feeling", "felt", "like", "just", "really",
    "know", "time", "people", "day", "going", "want"
})

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"^[a-z\s]", " ", text)
    words = text.split()

    words = [
        word for word in words
        if word not in custom_stopwords and len(word) > 2
    ]

    return words
```

GoEmotion Distribution of words per comment

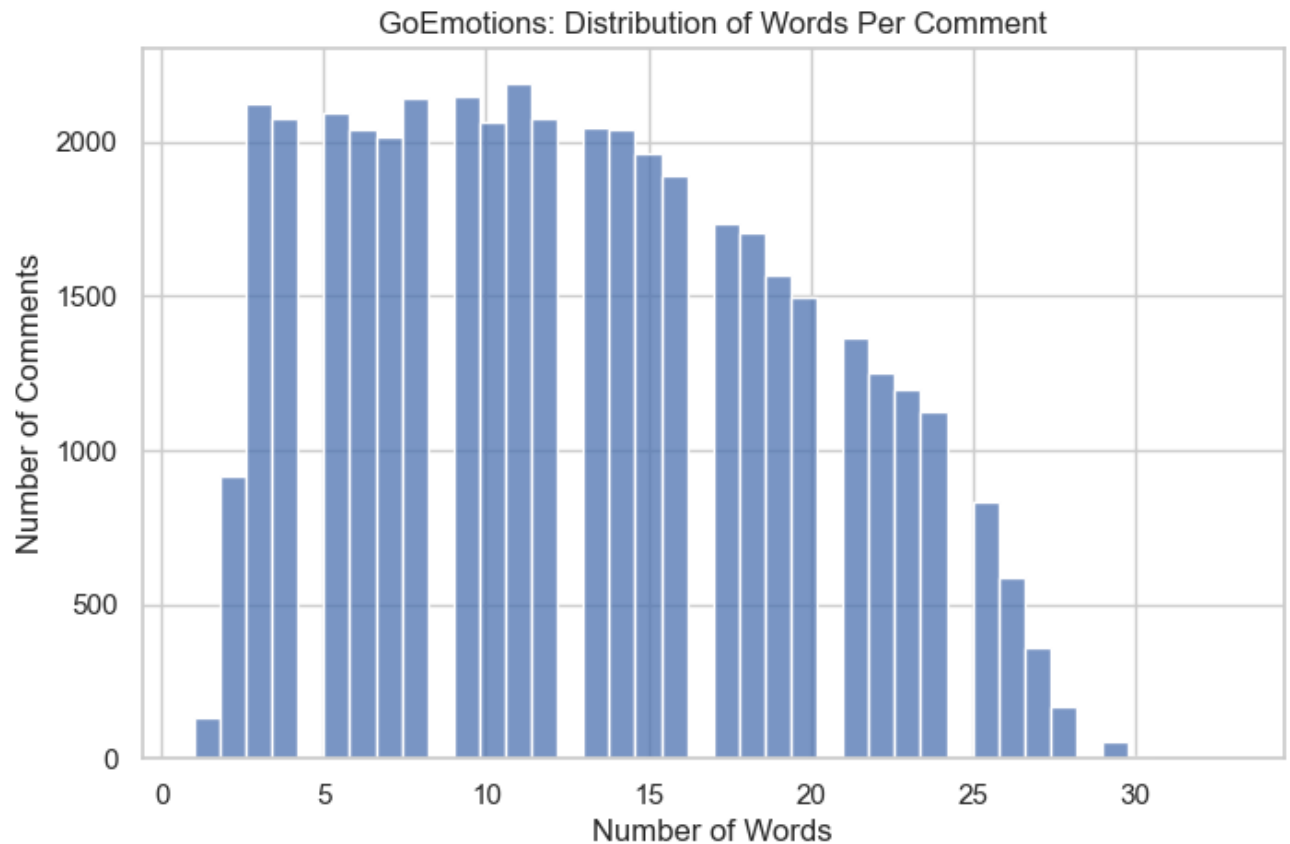
In [180...

```
# -----  
# 1. Basic Text Length Features  
# -----  
  
G_train["word_count"] = G_train["text"].apply(  
    lambda text: len(str(text).split())  
)  
  
G_train["clean_word_count"] = G_train["text"].apply(  
    lambda text: len(clean_text(text))  
)  
  
G_train["char_count"] = G_train["text"].apply(  
    lambda text: len(str(text))  
)  
  
print("Average raw words per comment:", G_train["word_count"].mean())  
print("Median raw words per comment:", G_train["word_count"].median())  
print("Average cleaned words per comment:", G_train["clean_word_count"].mean())  
print("Median cleaned words per comment:", G_train["clean_word_count"].median())  
print("Average characters per comment:", G_train["char_count"].mean())
```

Average raw words per comment: 12.840175074867542
Median raw words per comment: 12.0
Average cleaned words per comment: 5.196083851647086
Median cleaned words per comment: 5.0
Average characters per comment: 68.40087537433772

In [181...

```
plt.figure(figsize=(8,5))  
  
sns.histplot(  
    G_train["word_count"],  
    bins=40  
)  
  
plt.title("GoEmotions: Distribution of Words Per Comment")  
plt.xlabel("Number of Words")  
plt.ylabel("Number of Comments")  
plt.show()
```



GoEmotions Emotion vs Comment Length

```
In [182... length_rows = []

for emotion in G_emotion_cols:
    emotion_comments = G_train[G_train[emotion] == 1]

    length_rows.append({
        "Emotion": emotion,
        "Avg Raw Words": emotion_comments["word_count"].mean(),
        "Median Raw Words": emotion_comments["word_count"].median(),
        "Avg Cleaned Words": emotion_comments["clean_word_count"].mean(),
        "Median Cleaned Words": emotion_comments["clean_word_count"].median(),
        "Avg Characters": emotion_comments["char_count"].mean(),
        "Sample Count": len(emotion_comments)
    })

goemotion_length_table = pd.DataFrame(length_rows)

goemotion_length_table = goemotion_length_table.sort_values(
    "Avg Raw Words",
    ascending=False
)
```

goemotion_length_table

Out [182...

	Emotion	Avg Raw Words	Median Raw Words	Avg Cleaned Words	Median Cleaned Words	Avg Characters	Sample Count
20	optimism	15.537002	16.0	6.118912	6.0	80.512334	1581
22	realization	15.222523	15.0	5.818919	6.0	80.697297	1110
8	desire	14.622465	14.0	5.399376	5.0	73.263651	641
9	disappointment	14.449173	14.0	5.730496	5.0	77.160757	1269
6	confusion	14.409357	14.0	5.368421	5.0	75.980263	1368
12	embarrassment	14.306931	14.0	5.755776	5.0	77.415842	303
19	nervousness	14.262195	15.0	5.500000	5.0	75.439024	164
5	caring	14.108556	14.0	5.546458	5.0	74.111316	1087
10	disapproval	13.978239	14.0	5.652819	5.0	75.771513	2022
3	annoyance	13.964372	14.0	5.811336	5.0	75.713360	2470
24	remorse	13.792661	13.0	5.563303	5.0	72.102752	545
4	approval	13.697856	14.0	5.521266	5.0	73.187138	2939
14	fear	13.476510	13.0	5.411074	5.0	72.270134	596
23	relief	13.320261	13.0	5.339869	5.0	70.019608	153
7	curiosity	13.316750	13.0	4.995436	5.0	70.902328	2191
25	sadness	13.257919	13.0	5.337858	5.0	69.500000	1326
11	disgust	13.049180	13.0	5.587642	5.0	70.852459	793
17	joy	12.960744	12.5	5.316804	5.0	67.101240	1452
21	pride	12.873874	12.0	4.891892	5.0	66.072072	111
26	surprise	12.732075	12.0	5.201887	5.0	68.271698	1060
18	love	12.732023	12.0	5.069032	5.0	64.801055	2086
1	amusement	12.662371	12.0	5.483677	5.0	66.157216	2328
27	neutral	12.504185	12.0	5.016457	5.0	67.227091	14219
16	grief	12.116883	11.0	5.272727	5.0	63.935065	77
2	anger	11.971283	11.0	5.100191	5.0	64.691768	1567
0	admiration	11.913317	11.0	5.148668	5.0	64.293947	4130

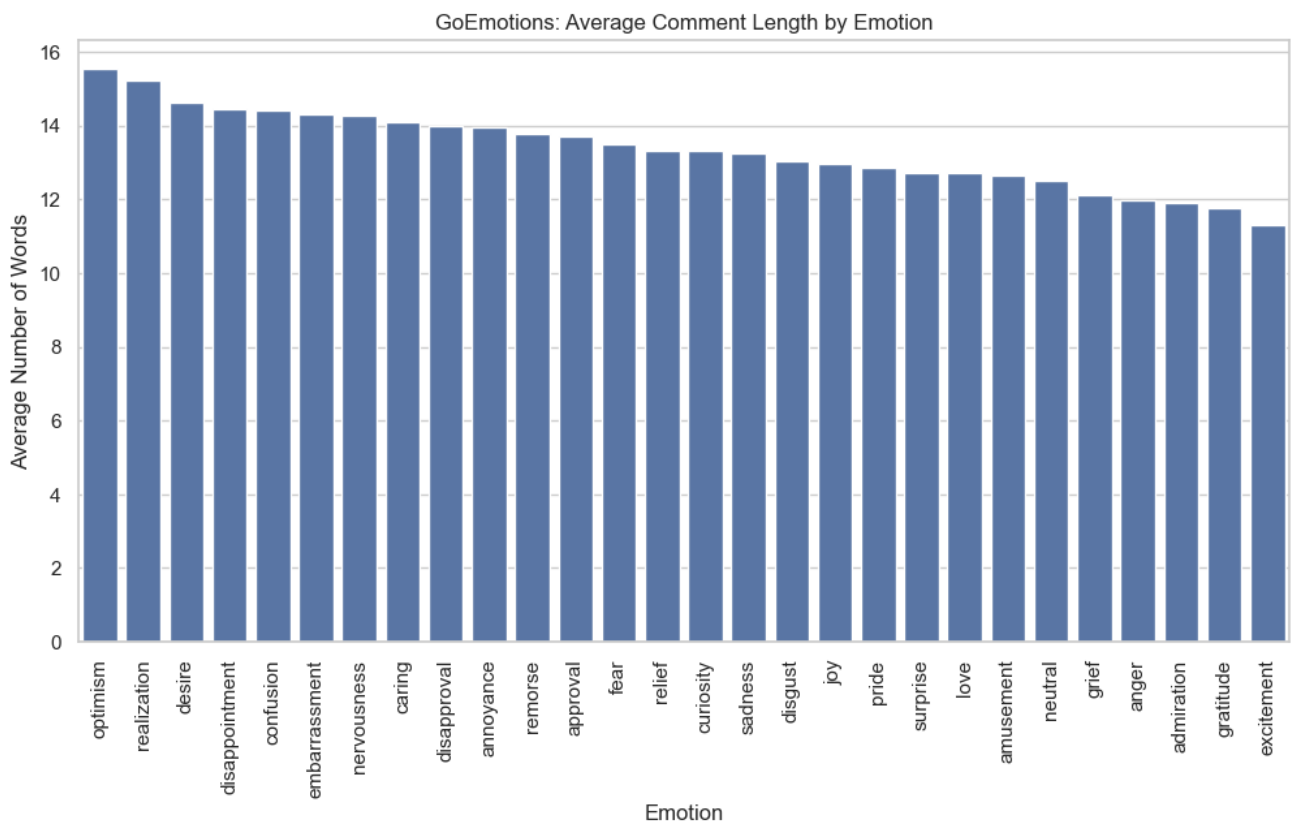
15	gratitude	11.761082	11.0	4.916604	4.0	62.588279	2662
13	excitement	11.303634	10.0	4.731536	4.0	59.908558	853

```
In [259... plt.figure(figsize=(12,6))

sns.barplot(
    data=goemotion_length_table,
    x="Emotion",
    y="Avg Raw Words"
)

#plt.ylim(10, 16)

plt.title("GoEmotions: Average Comment Length by Emotion")
plt.xlabel("Emotion")
plt.ylabel("Average Number of Words")
plt.xticks(rotation=90)
plt.show()
```



```
In [184... length_records = []

for emotion in G_emotion_cols:
    emotion_comments = G_train[G_train[emotion] == 1]
```

```

for count in emotion_comments["word_count"]:
    length_records.append({
        "Emotion": emotion,
        "Word Count": count
    })

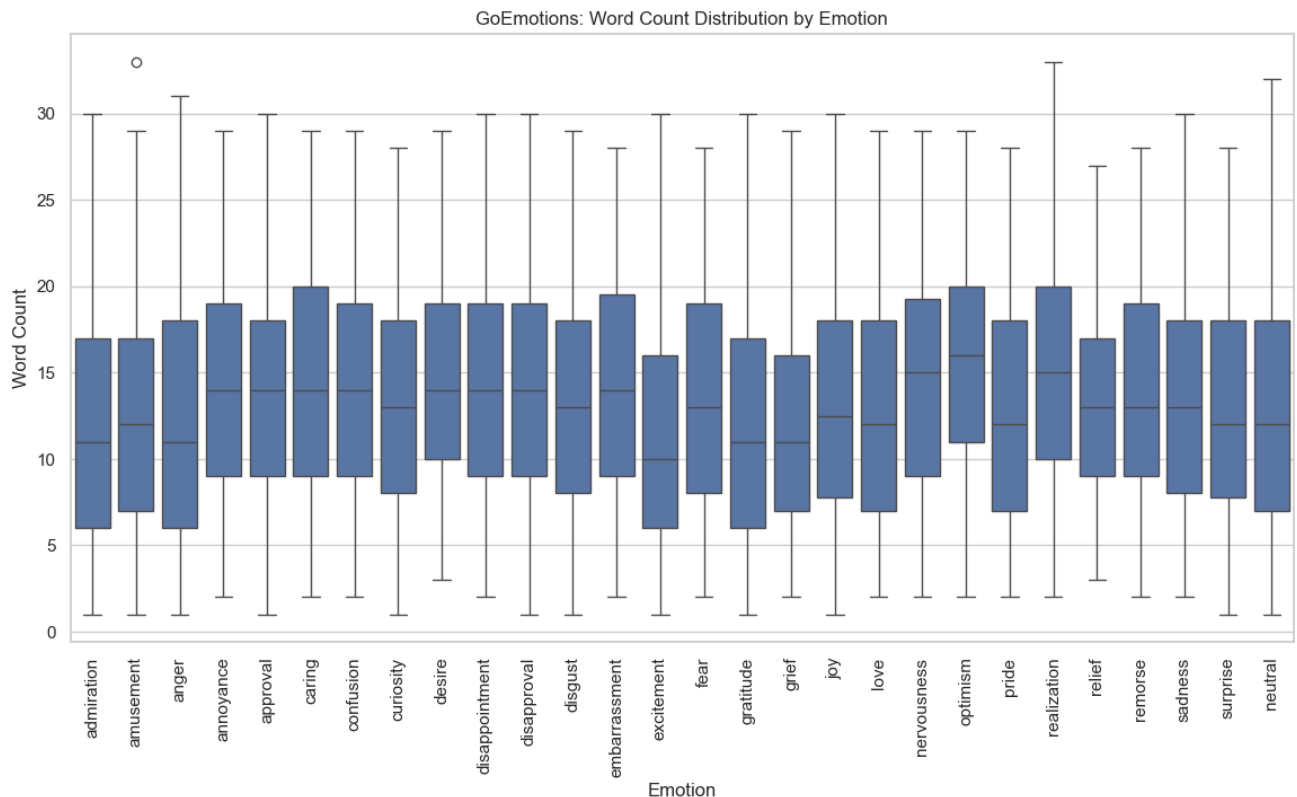
goemotion_length_long = pd.DataFrame(length_records)

plt.figure(figsize=(14,7))

sns.boxplot(
    data=goemotion_length_long,
    x="Emotion",
    y="Word Count"
)

plt.title("GoEmotions: Word Count Distribution by Emotion")
plt.xlabel("Emotion")
plt.ylabel("Word Count")
plt.xticks(rotation=90)
plt.show()

```



GoEmotion Label Distribution

In [185... *# Count how many labels each comment has*

```
G_train["num_labels"] = G_train[G_emotion_cols].sum(axis=1)

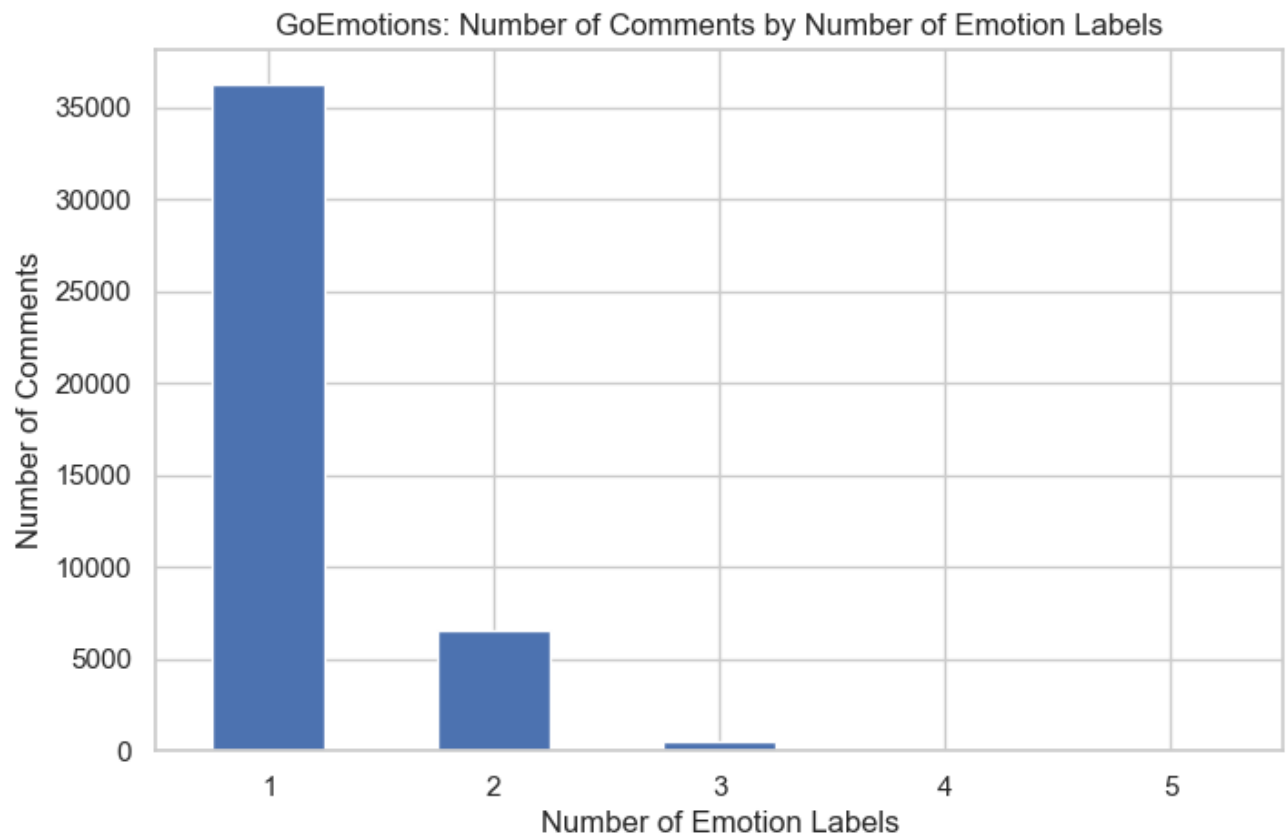
# Count how many comments have 1 label, 2 labels, 3 labels, etc.
label_count_distribution = (
    G_train["num_labels"]
    .value_counts()
    .sort_index()
)

print(label_count_distribution)

# Plot distribution
label_count_distribution.plot(kind="bar", figsize=(8, 5))

plt.title("GoEmotions: Number of Comments by Number of Emotion Labels")
plt.xlabel("Number of Emotion Labels")
plt.ylabel("Number of Comments")
plt.xticks(rotation=0)
plt.show()
```

```
num_labels
1    36308
2     6541
3      532
4        28
5         1
Name: count, dtype: int64
```

GoEmotions Dataset Statistics

```
In [186... print("Total samples in GoEmotions train:", len(G_train))
print("Total samples in GoEmotions test:", len(G_test))
print("Number of emotion columns:", 28)
G_labels_per_comment_train = G_train[G_emotion_cols].sum(axis=1)
G_labels_per_comment_test = G_test[G_emotion_cols].sum(axis=1)

print("Average labels per comment in GoEmotions train:", G_labels_per_comment_train.mean())
print("Average labels per comment in GoEmotions test:", G_labels_per_comment_test.mean())

multi_label_train = (G_labels_per_comment_train > 1).sum()
print("Multi-label samples in train data:", multi_label_train)
print("Percentage:", 100 * multi_label_train / len(G_train))

multi_label_test = (G_labels_per_comment_test > 1).sum()
print("Multi-label samples in train data:", multi_label_test)
print("Percentage:", 100 * multi_label_test / len(G_test))
```

Total samples in GoEmotions train: 43410
Total samples in GoEmotions test: 5427
Number of emotion columns: 28
Average labels per comment in GoEmotions train: 1.1772172310527529
Average labels per comment in GoEmotions test: 1.1662060070020268
Multi-label samples in train data: 7102
Percentage: 16.360285648468096
Multi-label samples in train data: 837
Percentage: 15.422885572139304

GoEmotions Emotion Frequency

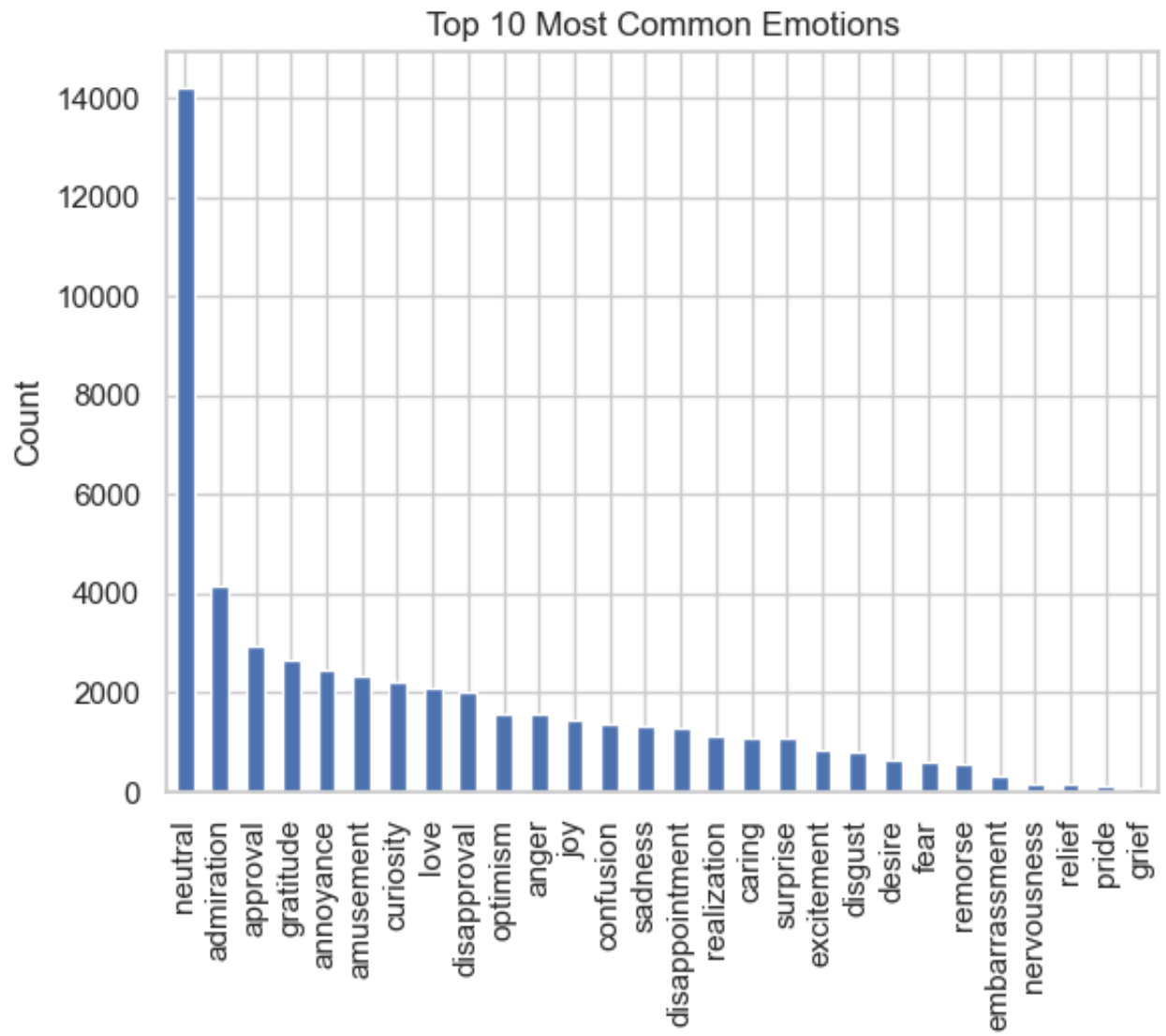
```
In [188... G_emotion_counts = G_train[G_emotion_cols].sum().sort_values(ascending=False)

G_emotion_counts.plot(kind="bar")
plt.title("Top 10 Most Common Emotions")
plt.ylabel("Count")
plt.show()

G_emotion_percentages = (
    G_emotion_counts / G_emotion_counts.sum() * 100
).round(2)

summary = pd.DataFrame({
    "Count": G_emotion_counts,
    "Percent": G_emotion_percentages
})

print(summary)
```



	Count	Percent
neutral	14219	27.82
admiration	4130	8.08
approval	2939	5.75
gratitude	2662	5.21
annoyance	2470	4.83
amusement	2328	4.56
curiosity	2191	4.29
love	2086	4.08
disapproval	2022	3.96
optimism	1581	3.09
anger	1567	3.07
joy	1452	2.84
confusion	1368	2.68
sadness	1326	2.59
disappointment	1269	2.48
realization	1110	2.17
caring	1087	2.13
surprise	1060	2.07
excitement	853	1.67
disgust	793	1.55
desire	641	1.25
fear	596	1.17
remorse	545	1.07
embarrassment	303	0.59
nervousness	164	0.32
relief	153	0.30
pride	111	0.22
grief	77	0.15

GoEmotions Emotion Co-occurrence

```
In [189... G_co_matrix = G_train[G_emotion_cols].T.dot(G_train[G_emotion_cols])
G_co_matrix_plot = G_co_matrix.copy()

for emotion in G_emotion_cols:
    G_co_matrix_plot.loc[emotion, emotion] = 0

G_co_matrix = G_co_matrix_plot
print(G_co_matrix)
```

	admiration	amusement	anger	annoyance	approval	caring	\
admiration	0	84	5	25	246	31	
amusement	84	0	15	40	57	8	
anger	5	15	0	269	8	9	
annoyance	25	40	269	0	37	13	
approval	246	57	8	37	0	65	
caring	31	8	9	13	65	0	
confusion	24	24	8	24	20	1	

curiosity	65	48	11	45	36	15
desire	33	10	4	12	18	11
disappointment	24	10	30	110	23	10
disapproval	14	20	43	178	19	14
disgust	10	9	53	80	13	2
embarrassment	3	8	6	9	1	2
excitement	75	13	0	4	25	2
fear	14	3	5	11	8	6
gratitude	279	82	11	20	62	24
grief	1	0	0	2	0	3
joy	126	90	4	6	41	23
love	192	55	7	12	60	41
nervousness	1	2	2	1	3	4
optimism	106	31	6	24	100	90
pride	23	1	1	2	5	0
realization	41	34	4	26	80	15
relief	4	3	2	4	7	6
remorse	9	10	3	8	12	14
sadness	16	18	17	26	10	29
surprise	53	13	10	9	14	3
neutral	94	57	73	132	202	61

	confusion	curiosity	desire	disappointment	...	love	\
admiration	24	65	33	24	...	192	
amusement	24	48	10	10	...	55	
anger	8	11	4	30	...	7	
annoyance	24	45	12	110	...	12	
approval	20	36	18	23	...	60	
caring	1	15	11	10	...	41	
confusion	0	212	2	16	...	10	
curiosity	212	0	11	21	...	25	
desire	2	11	0	12	...	27	
disappointment	16	21	12	0	...	10	
disapproval	22	31	5	54	...	16	
disgust	3	11	3	20	...	4	
embarrassment	2	10	0	10	...	1	
excitement	4	21	10	4	...	27	
fear	8	11	1	6	...	4	
gratitude	24	31	13	8	...	53	
grief	0	0	1	4	...	0	
joy	8	14	8	7	...	65	
love	10	25	27	10	...	0	
nervousness	2	3	0	7	...	2	
optimism	5	28	64	17	...	45	
pride	0	0	0	0	...	0	
realization	20	33	12	30	...	18	
relief	1	1	0	0	...	2	
remorse	5	14	2	15	...	4	
sadness	6	23	11	133	...	15	
surprise	23	57	1	11	...	9	

neutral	98	118	27	65	...	38
	nervousness	optimism	pride	realization	relief	remorse
\						
admiration	1	106	23	41	4	9
amusement	2	31	1	34	3	10
anger	2	6	1	4	2	3
annoyance	1	24	2	26	4	8
approval	3	100	5	80	7	12
caring	4	90	0	15	6	14
confusion	2	5	0	20	1	5
curiosity	3	28	0	33	1	14
desire	0	64	0	12	0	2
disappointment	7	17	0	30	0	15
disapproval	2	22	1	31	4	6
disgust	2	13	0	11	1	4
embarrassment	1	8	1	9	0	6
excitement	0	20	3	12	3	1
fear	34	7	0	6	0	2
gratitude	0	69	8	34	10	15
grief	1	0	0	0	0	3
joy	2	29	6	16	9	1
love	2	45	0	18	2	4
nervousness	0	1	0	3	1	0
optimism	1	0	10	31	4	17
pride	0	10	0	0	0	0
realization	3	31	0	0	1	6
relief	1	4	0	1	0	0
remorse	0	17	0	6	0	0
sadness	9	21	0	18	0	60
surprise	1	13	1	26	0	1
neutral	8	48	6	76	8	4
	sadness	surprise	neutral			
admiration	16	53	94			
amusement	18	13	57			
anger	17	10	73			
annoyance	26	9	132			
approval	10	14	202			
caring	29	3	61			
confusion	6	23	98			
curiosity	23	57	118			
desire	11	1	27			
disappointment	133	11	65			
disapproval	23	4	109			
disgust	11	0	29			
embarrassment	12	3	12			
excitement	1	40	21			
fear	15	9	20			
gratitude	17	22	14			

grief	22	2	2
joy	13	13	45
love	15	9	38
nervousness	9	1	8
optimism	21	13	48
pride	0	1	6
realization	18	26	76
relief	0	0	8
remorse	60	1	4
sadness	0	5	58
surprise	5	0	33
neutral	58	33	0

[28 rows x 28 columns]

```
In [190... pairs = []

for i in range(len(G_emotion_cols)):
    for j in range(i+1, len(G_emotion_cols)):
        pairs.append(
            (
                G_emotion_cols[i],
                G_emotion_cols[j],
                G_co_matrix.iloc[i, j]
            )
        )

pairs_df = pd.DataFrame(
    pairs,
    columns=["Emotion1", "Emotion2", "Count"]
)

pairs_df = pairs_df.sort_values(
    "Count",
    ascending=False
)

print(pairs_df.head(20))

mask = np.triu(np.ones_like(G_co_matrix, dtype=bool))

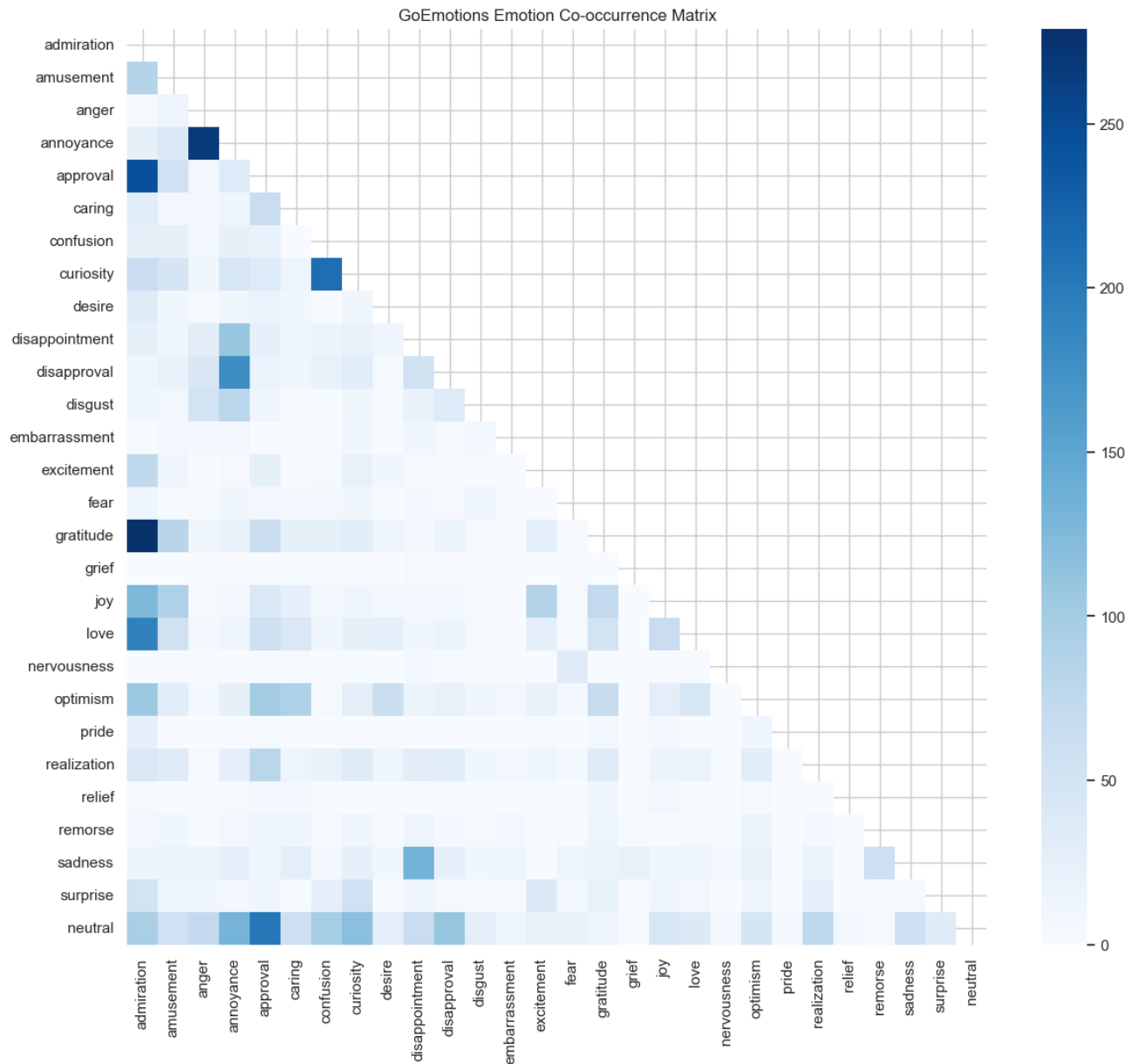
plt.figure(figsize=(14,12))

sns.heatmap(
    G_co_matrix,
    mask=mask,
    cmap="Blues"
)

plt.title("GoEmotions Emotion Co-occurrence Matrix")
```

```
plt.show()
```

	Emotion1	Emotion2	Count
14	admiration	gratitude	279
53	anger	annoyance	269
3	admiration	approval	246
147	confusion	curiosity	212
124	approval	neutral	202
17	admiration	love	192
84	annoyance	disapproval	178
222	disappointment	sadness	133
101	annoyance	neutral	132
16	admiration	joy	126
187	curiosity	neutral	118
83	annoyance	disappointment	110
241	disapproval	neutral	109
19	admiration	optimism	106
117	approval	optimism	100
167	confusion	neutral	98
26	admiration	neutral	94
139	caring	optimism	90
42	amusement	joy	90
276	excitement	joy	86



GoEmotions Gender Comparison

```
In [94]: G_male_comments = G_train[
    G_train["text"].str.contains(
        r"\b(he|him|his)\b",
        case=False,
        regex=True,
        na=False
    )
]

G_female_comments = G_train[
    G_train["text"].str.contains(
```

```

        r"\b(she|her|hers)\b",
        case=False,
        regex=True,
        na=False
    )
]

```

```

/var/folders/13/553h520n4_56sbjhxtryvs340000gn/T/ipykernel_72228/2565989487.
py:2: UserWarning: This pattern is interpreted as a regular expression, and
has match groups. To actually get the groups, use str.extract.
    G_train["text"].str.contains(
/var/folders/13/553h520n4_56sbjhxtryvs340000gn/T/ipykernel_72228/2565989487.
py:11: UserWarning: This pattern is interpreted as a regular expression, and
has match groups. To actually get the groups, use str.extract.
    G_train["text"].str.contains(

```

```

In [191]... G_male_emotions = G_male_comments[G_emotion_cols].sum().sort_values(ascending=True)

G_female_emotions = G_female_comments[G_emotion_cols].sum().sort_values(ascending=True)

G_male_pct = (
    G_male_comments[G_emotion_cols].sum()
    / len(G_male_comments)
    * 100
).sort_values(ascending=False)

G_female_pct = (
    G_female_comments[G_emotion_cols].sum()
    / len(G_female_comments)
    * 100
).sort_values(ascending=False)

G_comparison = pd.DataFrame({
    "Male %": G_male_pct,
    "Female %": G_female_pct
})

```

```

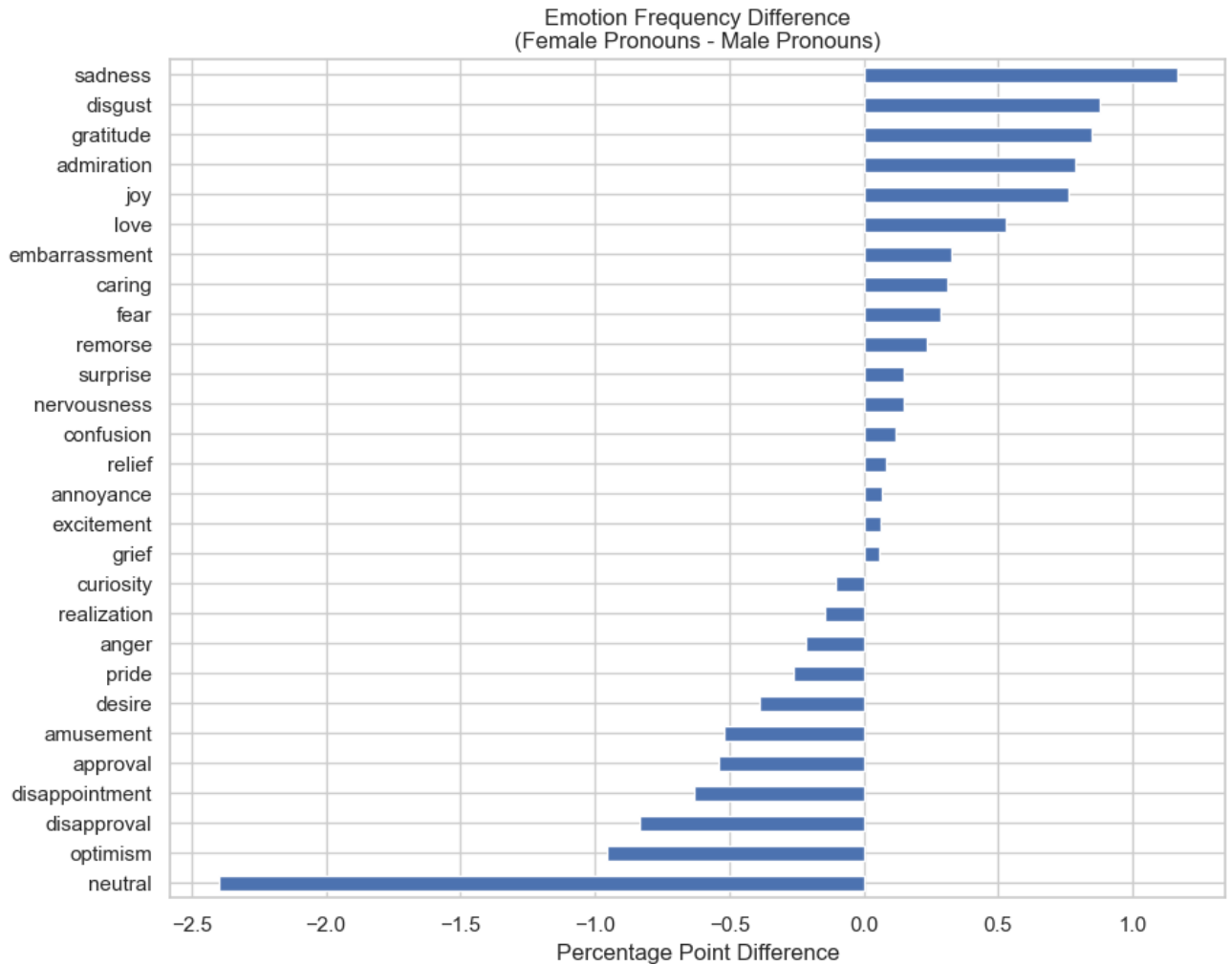
In [192]... G_comparison["Difference"] = (
    G_comparison["Female %"] - G_comparison["Male %"]
)

G_comparison = G_comparison.sort_values(
    "Difference",
    ascending=False
)

G_comparison["Difference"].sort_values().plot(
    kind="barh",
    figsize=(10,8)
)

```

```
plt.title("Emotion Frequency Difference\n(Female Pronouns - Male Pronouns)")  
plt.xlabel("Percentage Point Difference")  
plt.show()
```



In [100... G_comparison

Out[100...

	Male %	Female %	Difference
sadness	2.932895	4.100228	1.167332
disgust	1.853590	2.733485	0.879895
gratitude	2.252464	3.097950	0.845486
admiration	9.784139	10.569476	0.785337
joy	2.064758	2.824601	0.759843
love	5.349601	5.876993	0.527392
embarrassment	0.539653	0.865604	0.325951

caring	2.651337	2.961276	0.309938
fear	1.173158	1.457859	0.284701
remorse	0.586579	0.820046	0.233466
surprise	2.627874	2.779043	0.151169
nervousness	0.305021	0.455581	0.150560
confusion	3.026748	3.143508	0.116760
relief	0.281558	0.364465	0.082907
annoyance	5.490380	5.558087	0.067706
excitement	0.938527	1.002278	0.063751
grief	0.351947	0.410023	0.058075
curiosity	5.114969	5.011390	-0.103580
realization	2.557485	2.414579	-0.142906
anger	3.496011	3.280182	-0.215829
pride	0.305021	0.045558	-0.259463
desire	1.525106	1.138952	-0.386153
amusement	4.575317	4.054670	-0.520647
approval	5.959643	5.421412	-0.538231
disappointment	2.909432	2.277904	-0.631528
disapproval	4.434538	3.599089	-0.835449
optimism	4.692633	3.735763	-0.956869
neutral	39.347724	36.947608	-2.400116

GoEmotions Emotion vs Common Words

```
In [193... summary_rows = []

for emotion in G_emotion_cols:

    emotion_texts = G_train.loc[
        G_train[emotion] == 1,
        "text"
    ]
```

```

all_words = []

for text in emotion_texts:
    all_words.extend(clean_text(text))

top_words = [
    word
    for word, count in Counter(all_words).most_common(5)
]

summary_rows.append({
    "Emotion": emotion,
    "Most Common Words": top_words
})

pd.set_option("display.max_colwidth", None)

goemotion_word_table = pd.DataFrame(summary_rows)

goemotion_word_table

```

Out [193...

	Emotion	Most Common Words
0	admiration	[good, great, awesome, amazing, love]
1	amusement	[lol, haha, funny, fun, lmao]
2	anger	[fuck, hate, fucking, don, stupid]
3	annoyance	[don, fucking, stupid, fuck, damn]
4	approval	[right, good, agree, yes, yeah]
5	caring	[don, good, better, help, need]
6	confusion	[don, sure, think, confused, did]
7	curiosity	[did, curious, does, think, don]
8	desire	[wish, need, hope, love, wanted]
9	disappointment	[bad, don, didn, game, got]
10	disapproval	[don, think, doesn, bad, good]
11	disgust	[disgusting, worst, awful, weird, worse]
12	embarrassment	[shame, embarrassing, awkward, don, weird]
13	excitement	[excited, happy, wow, interesting, wait]
14	fear	[scared, terrible, afraid, scary, horrible]
15	gratitude	[thanks, thank, good, glad, sharing]

16	grief	[died, sorry, loss, rip, years]
17	joy	[happy, glad, enjoy, fun, good]
18	love	[love, loved, favorite, good, don]
19	nervousness	[worried, anxiety, nervous, makes, anxious]
20	optimism	[hope, good, luck, hopefully, better]
21	pride	[proud, pride, did, hope, man]
22	realization	[didn, thought, realize, think, don]
23	relief	[glad, thank, good, don, better]
24	remorse	[sorry, didn, regret, don, hope]
25	sadness	[sad, sorry, bad, sadly, don]
26	surprise	[wow, surprised, wonder, omg, didn]
27	neutral	[don, think, got, way, did]

GoEmotions Text Diversity and Lexical Richness

```
In [260... all_words = []

for text in G_train["text"]:
    all_words.extend(clean_text(text))

G_vocab_size = len(set(all_words))
G_total_words = len(all_words)
G_lexical_richness = G_vocab_size / G_total_words

G_dataset_diversity = pd.DataFrame({
    "Metric": [
        "Total Words",
        "Vocabulary Size",
        "Lexical Richness"
    ],
    "Value": [
        G_total_words,
        G_vocab_size,
        G_lexical_richness
    ]
})

G_dataset_diversity
```

Out [260...

	Metric	Value
0	Total Words	225562.000000
1	Vocabulary Size	24921.000000
2	Lexical Richness	0.110484

In [262...

```

G_diversity_rows = []

for emotion in G_emotion_cols:

    emotion_texts = G_train.loc[
        G_train[emotion] == 1,
        "text"
    ]

    words = []

    for text in emotion_texts:
        words.extend(clean_text(text))

    total_words = len(words)
    unique_words = len(set(words))

    lexical_richness = (
        unique_words / total_words
        if total_words > 0 else 0
    )

    G_diversity_rows.append({
        "Emotion": emotion,
        "Total Words": total_words,
        "Vocabulary Size": unique_words,
        "Lexical Richness": lexical_richness
    })

G_diversity_table = pd.DataFrame(G_diversity_rows)

G_diversity_table = G_diversity_table.sort_values(
    "Lexical Richness",
    ascending=False
)

G_diversity_table

```

Out [262...

	Emotion	Total Words	Vocabulary Size	Lexical Richness
21	pride	543	391	0.720074

16	grief	406	282	0.694581
19	nervousness	902	622	0.689579
23	relief	817	544	0.665851
12	embarrassment	1744	1032	0.591743
8	desire	3461	1810	0.522970
14	fear	3225	1623	0.503256
11	disgust	4431	2186	0.493342
24	remorse	3032	1457	0.480541
13	excitement	4036	1861	0.461100
22	realization	6459	2887	0.446973
26	surprise	5514	2452	0.444686
6	confusion	7344	3193	0.434777
9	disappointment	7272	3077	0.423130
2	anger	7992	3351	0.419294
7	curiosity	10945	4408	0.402741
25	sadness	7078	2816	0.397853
5	caring	6029	2309	0.382982
10	disapproval	11430	4372	0.382502
17	joy	7720	2855	0.369819
18	love	10574	3766	0.356157
3	annoyance	14354	5052	0.351958
20	optimism	9674	3356	0.346909
4	approval	16227	5406	0.333148
1	amusement	12766	4105	0.321557
0	admiration	21264	5809	0.273185
15	gratitude	13088	3509	0.268108
27	neutral	71329	14993	0.210195

```
In [263... plt.figure(figsize=(10,8))
sns.barplot(
```

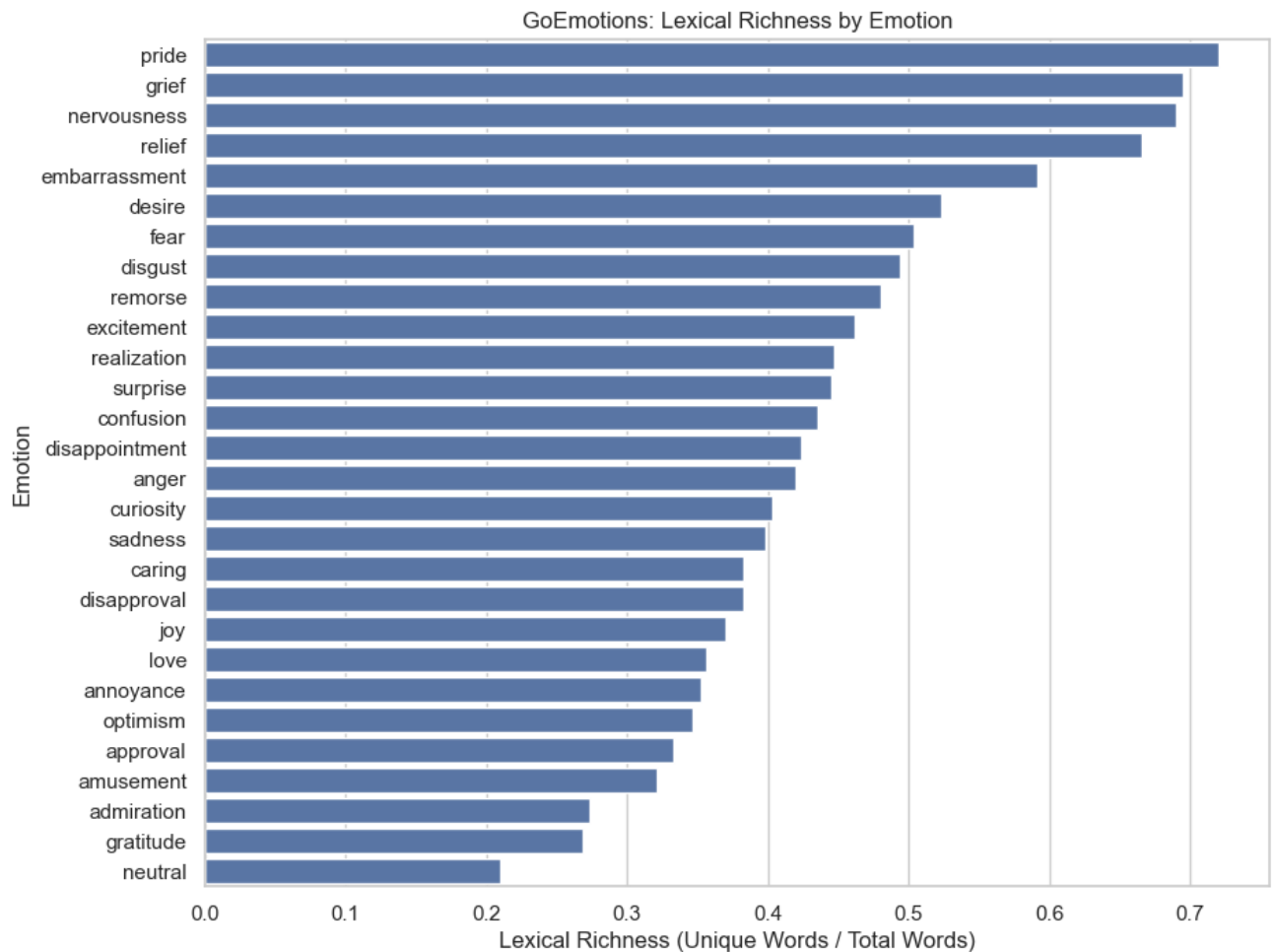


```

data=G_diversity_table,
y="Emotion",
x="Lexical Richness"
)

plt.title("GoEmotions: Lexical Richness by Emotion")
plt.xlabel("Lexical Richness (Unique Words / Total Words)")
plt.ylabel("Emotion")
plt.show()

```



```

In [265... plt.figure(figsize=(10,8))

sns.barplot(
    data=G_diversity_table,
    y="Emotion",
    x="Vocabulary Size"
)

plt.title("GoEmotions: Vocabulary Size by Emotion")
plt.xlabel("Vocabulary Size")
plt.ylabel("Emotion")
plt.show()

```

